

MATHEMATICAL CONCEPTS FOR MACHINE LEARNING–
AASD-4001

TERM PROJECT

CLASSIFICATION OF HEART DISEASES

GROUP ONE

TEAM MEMBERS

Bikash Giri (101575097)
Chia-Wei Chang (101570243)
Diparshan Bhattarai (101494737)
Hsi-Teng Liu (101576074)
Gavriel Kirichenko (101119609)
Abdul-Rasaq Omisesan (101571156)
Callum Arul (101585383)
Friha Hussainyar (101591222)
Vivek Panchal (101484976)

Applied AI Solutions Development, School of Computer Technology
George Brown College – Toronto, ON

PROF. LOSHANAH SHAKTHIVEL

January 28, 2025

Table of Contents

Table of Contents1

1. Introduction.....	3
Purpose	3
Problem Statement	4
Python Version & Dependencies.....	4
Dataset Overview	4
Attributes	4
Exploratory Data Analysis.....	5
3. Data Preprocessing	5
1. Introduction.....	5
2. Data Cleaning.....	5
2.1 Handling Missing Values	5
2.2 Handling Duplicate Records.....	5
3. Data Transformation.....	6
3.1 Encoding Categorical Variables.....	6
3.2 Scaling and Normalization	6
4. Handling Outliers	6
5. Data Splitting	6
6. Conclusion	6
7. Feature Selection Analysis	6
Decisions prior to model building	7
4. Model Building	7
4.1 Logistic Regression	7
Models Implemented.....	7
Observation and Analysis	8
4.2 Decision Tree.....	9
Introduction	9
Observations and Analysis.....	9
Key Findings	9
Conclusion	10
4.3 Random Forest.....	10
Introduction.....	10
Objective	10
Four models were tested using confusion matrices:	10
Overfitting Challenges	10
Hyperparameters Tuning and Its Role	10
Findings	11

Model Accuracy Comparison	11
Results and Evaluation	11
4.4 Stochastic Gradient Descent (SGD).....	11
Introduction.....	11
Model Implementation	11
Baseline Model	12
Hyperparameter Tuning	12
Tuned Parameters:	12
Model Performance	12
Observations:	12
Confusion Matrix Analysis	13
Learning Curve Analysis.....	13
Conclusion for SGD.....	14
4.5 Support Vector Machine (SVM).....	14
Introduction.....	14
Training Metrics	14
Models	14
Hyperparameters	14
Observations and Analysis.....	15
5. Conclusion	15

1. Introduction

Purpose

Heart disease remains one of the leading causes of mortality worldwide, making it crucial to develop effective analytical and classification models for early detection and diagnosis. This project focuses on analyzing and classifying heart disease data using machine learning techniques. By leveraging a well-established dataset from Kaggle, we aim to explore patterns,

optimize predictive mode, and evaluate their effectiveness in identifying heart disease cases. The project follows a structured workflow, incorporating data preprocessing, feature selection, model evaluation, and impact assessment. Additionally, version control mechanisms ensure seamless collaboration among team members.

Problem Statement

Cardiovascular diseases, particularly heart disease, pose significant health risks worldwide. The ability to predict heart disease based on clinical attributes can aid in early diagnosis, personalized treatment, and improved patient outcomes. However, effectively classifying and analyzing this condition remains a challenge due to data variability and complex feature interactions. This project aims to build a robust analytical pipeline to process heart disease data, extract meaningful insights, and develop machine learning models that enhance prediction accuracy.

Python Version & Dependencies

The project is built using **Python (>=3.12)**, ensuring compatibility with modern machine-learning tools and libraries. The dependencies required for the analysis and classification tasks include:

- ipykernel>=6.29.5
- matplotlib>=3.10.0
- numpy>=2.2.1
- pandas>=2.2.3
- scikit-learn>=1.6.1
- seaborn>=0.13.2
- lightgbm>=4.5.0

These dependencies support data analysis, visualization, preprocessing, and machine learning model development.

Dataset Overview

The data is a subset of the original data collected in 1988 in Cleveland, Hungary, Switzerland, and Long Beach V. This subset of 14 attributes is what most publish experiments use.

Attributes

- Age
- Sex
- Chest pain type (4 values)
- Resting blood pressure
- Serum cholestoral in mg/dl
- Fasting blood sugar > 120 mg/dl
- Fasting electrocardiographic results (values 0,1,2)
- Maximum heart rate achieved.
- Exercise induced angina.
- Oldpeak = ST depression induced by exercise relative to rest
- The slope of the peak exercise ST segment
- Number of major vessels (0-3) colored by flourosopy
- thal: 0 = normal; 1 = fixed defect; 2 = reversable defect

Exploratory Data Analysis

In the EDA, we use various types of plots like box plots and bar charts to determine the distribution and variance between features like age, gender, clinical factors and medical risk factors. We identified the following important observations:

1. The dataset is balanced between positive (51.32%) and negative cases (48.68%).
2. It is skewed more male (69.56%) vs female (30.44%).
3. In this dataset, people with heart disease tend to be younger than those without. This is especially true for males (lower median in the box plot).
4. Clinical features like trestbps (resting blood pressure) and cholesterol do not show much variation between disease and no disease groups. However, people with disease tend to have higher thalach (max heart rate) and lower oldpeak (ST depression induced by exercise relative to rest) than those without.
5. Analysis of medical risk factors identified which specific types of categorical variables were more associated with heart disease. This includes:
 - typical angina (CP=0)
 - no exercise induced angina (exang = 0)
 - flat ST slope (ST = 0)
 - 0 major vessels colored by fluoroscopy (ca = 0)
 - fixed defect thalassemia (thal = 2)

3. Data Preprocessing

1. Introduction

Data preprocessing is a crucial step in data analysis and machine learning, ensuring that the dataset is clean, well-structured, and suitable for analysis. This report outlines the preprocessing steps applied to the heart disease dataset to enhance data quality and model performance.

2. Data Cleaning

2.1 Handling Missing Values

The dataset was checked for missing values. No missing data was found, but if any had been detected, we would have handled it using one of the following methods:

- Removing rows or columns with excessive missing values.
- Imputing missing values using mean, median, or mode, depending on the feature type.

2.2 Handling Duplicate Records

Duplicate records can distort the analysis. The dataset was scanned for duplicates, and any found were removed to maintain data integrity.

3. Data Transformation

3.1 Encoding Categorical Variables

Some features in the dataset, such as 'Sex', 'Chest Pain Type (cp)', and 'Thalassemia (thal)', were categorical. These were encoded into numerical values using OneHotEncoding (get_dummies function).

3.2 Scaling and Normalization

Numerical features such as 'Age', 'Resting Blood Pressure (trestbps)', and 'Cholesterol (chol)' had different ranges, which could affect model performance. We applied:

- **Standardization** (StandardScaler()) for models that assume normally distributed data.

4. Handling Outliers

Outliers in continuous variables like 'Cholesterol' and 'Max Heart Rate (thalach)' were detected using box plots and Z-score analysis. Identified outliers were treated by:

- Using the IQR Method

5. Data Splitting

The dataset was split into training and testing sets using an 70-30 ratio to ensure model generalization. A stratified sampling approach was used to maintain the class distribution in the training and test sets.

6. Conclusion

The preprocessing steps ensured that the dataset was clean, consistent, and suitable for analysis. These improvements helped enhance the quality of the dataset, making it more reliable for building predictive models.

7. Feature Selection Analysis

To identify the most important predictors of heart disease in our dataset, we applied multiple feature-selection methods—namely Pearson correlation, Chi-Square tests, Recursive Feature Elimination (RFE) with Logistic Regression, Logistic Regression (L1 regularization), Random Forest feature importance, and LightGBM feature importance. Each method independently “voted” on whether a given feature was relevant; the votes were then summed to produce a final ranking. We configured each algorithm to pick its top five features, and the Feature Selector tallied the votes for each feature across all methods.

Initially, five features emerged as the most influential:

1. oldpeak (ST depression)
2. thalach (maximum heart rate achieved)
3. cp (chest pain type)
4. ca (number of major vessels colored by fluoroscopy)
5. thal (thalassemia)

Moreover, to avoid overfitting in decision tree and random forest models, consulting pharmacists, and leveraging domain knowledge, we noted a discrepancy for oldpeak and ca. From a clinical perspective, higher values of these two features should correlate positively with an increased likelihood of heart disease. Yet, our exploratory data analysis indicated a negative correlation. Possible reasons for this mismatch include data collection inconsistencies or unique clinical variations not reflected in the dataset.

To mitigate potential overfitting and maintain interpretability, we decided to remove oldpeak and ca from our final feature set. By aligning machine learning outputs with established medical insights, we aim to enhance both model generalizability and clinical relevance. Consequently, our final recommended features for model building were:

1. thalach (maximum heart rate achieved)
2. cp (chest pain type)
3. thal (thalassemia)

This streamlined set balances strong empirical performance (as revealed by the voting process) with credible domain knowledge, providing a more robust foundation for subsequent model training and evaluation.

Decisions prior to model building

We wanted an assortment of models to focus on different metrics. In heart disease classification, it is dangerous to miss-classify someone as no disease when they do in fact have the disease. They may not receive timely treatment. Hence, it is important to optimize for recall to minimize false negatives. However, this may come with trade-offs, so we decided to have some models tuned for accuracy and some for recall (logistic regression and SGD).

4. Model Building

4.1 Logistic Regression

Models Implemented

1. **Baseline Model:**
 - a. Trained with default hyperparameters and no feature selection.
 - b. Serves as a benchmark for comparison.
2. **Tuned Model:**
 - a. Hyperparameters were optimized using **GridSearchCV** to improve performance.
 - b. Focused on maximizing recall to reduce false negatives.
3. **Feature-Selected Model:**
 - a. Only the most relevant features were selected based on their correlation with the target variable.
 - b. Aimed to improve model efficiency and performance.
4. **Tuned Feature-Selected Model:**
 - a. Combined hyperparameter tuning with feature selection for optimal results.

Observation and Analysis

Model	Accuracy	Precision	Recall	F1-Score	Class
Baseline	0.8517241379310345	0.83 0.88	0.89 0.81	0.86 0.84	1 0
Tuned	0.7103448275862069	0.64 0.98	0.99 0.42	0.78 0.59	1 0
Feature-selected	0.8448275862068966	0.84 0.85	0.86 0.83	0.85 0.84	1 0
Tuned Feature-Selected	0.7379310344827587	0.67 0.93	0.96 0.51	0.79 0.66	1 0

1. **Baseline Model:**
 - a. Achieved the highest accuracy (**85.17%**) with balanced precision (0.83) and recall (0.89).
 - b. Best overall performance for general use.
2. **Tuned Model:**
 - a. Achieved the highest recall (**0.99**) but suffered in precision (0.64) and accuracy (71.03%).
 - b. Useful in scenarios where missing a heart disease case is critical, but false positives are acceptable.
3. **Feature-Selected Model:**
 - a. Performed almost as well as the baseline (84.48% accuracy) but with slightly lower recall (0.86).
 - b. Demonstrates that feature selection can improve efficiency without significantly sacrificing performance.
4. **Tuned Feature-Selected Model:**
 - a. Combined tuning and feature selection but resulted in lower accuracy (73.79%) and imbalanced precision/recall.
 - b. Not recommended for this dataset.

The **baseline logistic regression model** is a good balanced choice for heart disease prediction in this project. It achieves the highest accuracy (85.17%) and maintains a good balance between precision (0.83) and recall (0.89). However, **the tuned models** excel in recall (99%), so these might be more suited for the case of heart disease classification to minimize false negatives. Feature selection slightly improved efficiency but did not outperform the baseline.

4.2 Decision Tree

Introduction

In this section different models of decision tree with varying configurations, baseline model, model with hyperparameter tuning and models with feature selection are evaluated. The objective is to determine the most balanced model that provides both high accuracy and generalizability.

Observations and Analysis

Model	Accuracy	Precision	Recall	F1-Score	Class
Baseline (No FS, No GS)	0.9896	0.98 1.00	1.00 0.98	0.99 0.99	0 1
Tuned (With GridSearch)	0.86	0.82 0.92	0.93 0.79	0.87 0.85	0 1
Feature-selected (No GridSearch)	0.89	0.85 0.93	0.94 0.84	0.89 0.88	0 1
Tuned Feature- Selected (Both)	0.81	0.84 0.79	0.77 0.86	0.80 0.82	0 1

0 --> No Disease | 1 --> Disease

- **Baseline Model** – achieved the highest accuracy (0.9897) but exhibited clear signs of overfitting.
- Baseline Model with Grid Search – Reduced overfitting by optimizing hyperparameters but had a lower accuracy (0.8621).
- Model with Feature Selection (No GridSearch) - Improved efficiency by removing less relevant features and achieved an accuracy of 0.8897.
- Model with Feature Selection and GridSearch – Most generalizable model with an accuracy of 0.8138, balancing performance and overfitting.

Key Findings

- Highest accuracy was seen in the baseline mode, but the absence of false positives and minimal false negatives show overfitting, making it unreliable for unseen data.
- GridSearch helped reduce overfitting but resulted in lower accuracy, providing more realistic assessment of the model performance.
- Feature selection improved efficiency by eliminating irrelevant features, leading to slightly better accuracy compared to GridSearch alone.

- The model with both feature selection and GridSearch exhibited the lowest accuracy but showed the highest generalizability, making it more suitable for real-world application.
- Given that the dataset is related to the heart disease prediction, ***high recall is prioritized*** to minimize false negatives, as misclassifying a patient without heart disease when they have it can have severe consequences.

Conclusion

The **Model with Feature Selection and GridSearch** is the **most suitable** as it balances between accuracy and generalizability. Furthermore, prioritizing recall helps in minimizing false negatives, making this model a more reliable choice for heart disease prediction.

4.3 Random Forest

Introduction

Random Forest is a Supervised machine learning algorithm that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting. It is used for classification tasks. However, it can still overfit if not finely tuned.

Objective

This analysis aimed to evaluate the impact of feature selection and hyperparameter tuning on Random Forest. Specifically, we assessed whether GridSearchCV improves generalization and whether removing irrelevant features helps prevent overfitting.

Four models were tested using confusion matrices:

1. Model 1 (Baseline: No Feature Selection, No GridSearchCV)
2. Model 2 (Baseline: No Feature Selection, With GridSearchCV)
3. Model 3 (With Feature Selection, No GridSearchCV)
4. Model 4 (With Feature Selection, With GridSearchCV)

Each model was evaluated based on its ability to classify instances of "Disease" vs. "No Disease."

Overfitting Challenges

We experimented extensively with RandomForestClassifier, but the model continued to overfit, suggesting that there may be underlying issues with the dataset itself. The dataset may have features that allow the model to memorize patterns too well rather than generalizing them. To address this, we focused on tuning max_depth, which controls the depth of each tree. A deeper tree allows the model to capture more complexity but also risks overfitting. To counteract this, we tested multiple values for max_depth, ranging from shallow trees (depth=2) to more complex ones (depth=5), to see its impact on performance.

Hyperparameters Tuning and Its Role

GridSearchCV was employed to optimize hyperparameters, focusing on max_depth, n_estimators, and min_samples. These adjustments aimed to reduce overfitting while keeping high accuracy and recall.

Findings

Model Accuracy Comparison

Base Model (No Feature Selection, No GridSearchCV)

- Training Accuracy: 1.0
- Test Accuracy: 1.0

Tuned Model (No Feature Selection, With GridSearchCV)

- Training Accuracy: 0.9392
- Test Accuracy: 0.9138

Feature Selected Model (With Feature Selection, No GridSearchCV)

- Training Accuracy: 0.9466
- Test Accuracy: 0.8931

Tuned Feature Selected Model (With Feature Selection, With GridSearchCV)

- Training Accuracy: 0.8680
- Test Accuracy: 0.8172

Results and Evaluation

- The Base Model significantly overfits, achieving perfect accuracy on both training and test sets.
- Applying GridSearchCV (Tuned Model) improved generalization, reducing the training accuracy slightly while maintaining a high-test accuracy.
- Feature Selection (Feature Selected Model) reduced overfitting slightly, but test accuracy also dropped.
- Combining Feature Selection with GridSearchCV (Tuned Feature Selected Model) resulted in the most balanced model, reducing overfitting the most, though at the cost of lower accuracy.

This suggests that hyperparameter tuning is more effective in reducing overfitting than feature selection alone. However, feature selection combined with tuning provides the best trade-off between accuracy/recall and generalization.

4.4 Stochastic Gradient Descent (SGD)

Introduction

The SGD Classifier which is the focus of this section is a linear model trained via gradient descent optimization. It is very advantageous for datasets and data with many attributes. This report put to the test the operational characteristics of an SGD classifier in the context of heart disease prognosis with an emphasis on minimizing false negatives with recall optimization.

Model Implementation

The SGD classifier was implemented using the SGDClassifier class from Scikit-learn, with variations of hyperparameter choices and optimization strategies being tried out.

Baseline Model

- Loss function: Hinge (equivalent to a linear SVM)
- Penalty: L2 regularization
- Learning rate: Constant
- Performance evaluated using accuracy, precision, recall, and F1-score.

Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV for the improvement of the model's predictive performance, with the primary aim of maximizing recall (scoring value was set to “recall”) while sustaining a fair balance with precision.

Tuned Parameters:

- Loss function: modified_huber and log loss
- Penalty: L2 and elastic net
- Alpha (regularization strength): 0.0001, 0.001, 0.01 (lower regularization for better recall)
- Learning rate: Adaptive
- Number of iterations (epochs): 1000

Model Performance

We compared three models: the baseline SGD classifier, the hyperparameter-tuned SGD classifier, and the feature-selected SGD classifier.

Model	Accuracy	Precision	Recall	F1-Score
Baseline	0.845	0.848	0.842	0.845
Tuned	0.824	0.765	0.938	0.843
Feature-selected	0.690	0.841	0.473	0.605
Tuned Feature-Selected	0.828	0.776	0.925	0.844

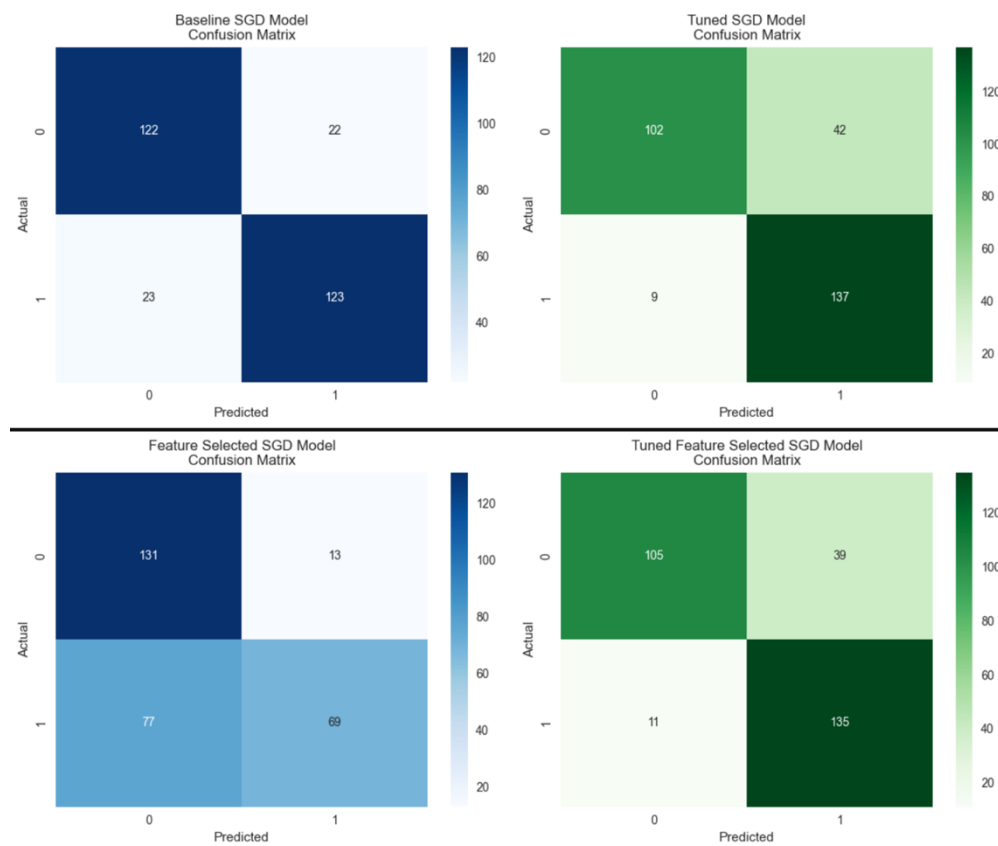
Observations:

- An improvement in recall was attained by tuning the model, from 0.842 to 0.938, thereby becoming a better candidate for medical diagnosis as false negatives pose dire consequences.

Group One Term Project

- Down the line, some decrease in accuracy was observed in the recall-vs.-precision compromise.
- Furthermore, when feature selection is carried out without tuning, there is a drop in performance, indicating that all the features are meaningfully contributing to the prediction.

Confusion Matrix Analysis



Confusion matrices for each of the models demonstrate some classic

precision/recall trade-offs:

- The tuned model reduced false negatives nicely, in line with its objective of maximizing recall.
- The feature-selected model had more misclassifications, supporting the decision to retain all features.

Learning Curve Analysis

The cross-validation learning curves demonstrate that the SGD classifier overfits for small training sizes but generalizes better for larger data. For larger datasets, the training score and cross-validation get closer, probably showing potential benefit from more data. However, it does indicate that overfitting is not a problem.

Conclusion for SGD

The SGD classifier, especially the tuned version, works well for this problem because it is computationally efficient and optimizes recall. The tuned model and tuned feature selected model are both good choices for this problem.

This analysis provides useful insights into optimizing SGD classifiers for applications focused on recall, making them useful in medical and high-risk decision-making tasks.

4.5 Support Vector Machine (SVM)

Introduction

We experimented on the Support Vector Classifier using the `sklearn.svm.SVC` class from the Scikit Learn library. We compare the base model with different features and hyperparameters of various levels of regularizations, kernel functions and kernel coefficients(gamma).

Training Metrics

Model	Accuracy	Precision	Recall	F1-Score	Cls	Kernel	C
Baseline	0.92	0.91 0.94	0.95 0.90	0.93 0.92	1 0	RBF	1.0
Tuned	0.99	1.00 0.98	0.98 1.00	0.99 0.99	1 0	Poly	0.1
Feature-Selected	0.88	0.87 0.88	0.88 0.87	0.88 0.87	1 0	RBF	1.0
Feature-Selected Tuned	0.98	0.97 1.00	1.00 0.97	0.98 0.98	1 0	RBF	1000

Models

The models are separated into 2 groups by different training features. One group is trained with the features that were picked with correlation; the other group is trained with the features picked through feature selection analysis methods. Each group holds two models: one trained with default parameters; the other with parameters tuned using `sklearn.model_selection.GridSearchCV`.

Hyperparameters

- C (Regularization)
0.1, 1, 10, 100, 1000
- Kernel
RBF, Sigmoid, Polynomial

- Gamma (Kernel Coefficient)
1, 0.1, 0.01, 0.001, 0.0001

Observations and Analysis

All models perform exceptional well that are above expectations. All hyperparameter tuned models achieve nearly 100% accuracy. This leads to concerns of overfitting, so we compare the accuracy scores of testing and training datasets. We found that the high accuracy scores are consistent with each other so that it is possible the models are not overfitting.

	Testing Accuracy	Training Accuracy
Baseline	0.924138	0.925816
Baseline Tuned	0.989655	1.000000
Feature Selected	0.875862	0.897626
Feature Selected Tuned	0.875862	0.985163

The reason the model performs so well can be a few features have correlation with the label so high that a mere statistical analysis might be enough to predict accurately.

The Feature-Selected Tuned model is strongly regularized with the C value of 1000. The regularization might push the model to overfitting, further experiments with lower values should be tried.

The polynomial kernel function protracted the training process, max_iter had to be set to 1000 to limit the iterations, otherwise it would take too long for such a small sample size. While RBF kernel produces similar performance to polynomial kernel, it is much efficient. For exploratory training could use RBF kernel instead of polynomial kernel.

5. Conclusion

For the problem of heart disease, we have an assortment of models to choose from. The best models seem to be Random Forest and Decision Tree with near perfect accuracy with baseline models. However, this may benefit from more testing on whether these are good on unseen data but it is important to note that this dataset was very clean and balanced, so these near perfect results are a possibility. The SVM model is close behind offering similar results. The Logistic Regression and SVD models do not have as high accuracy but were tuned specifically for recall and perform well there. We also conducted cross validation tests and plotted the learning curves and determined that overfitting was not a problem here. These models were also sensitive to feature removal and the performance dropped quite a bit in the feature-selected models.

Moving forward, based on the results, we believe that using multiple different models here, some optimized for accuracy and some for recall, followed by a voting procedure, would likely have the most balanced and safe results for real-world predictions. However, with more testing, like determining generalizability and robustness, we may be able to converge on a single best performing model. This project thus showcases how data can be used to classify heart disease, what considerations we need to make when considering medical data and how certain models perform better at certain tasks.